26/2/25 DAY 3

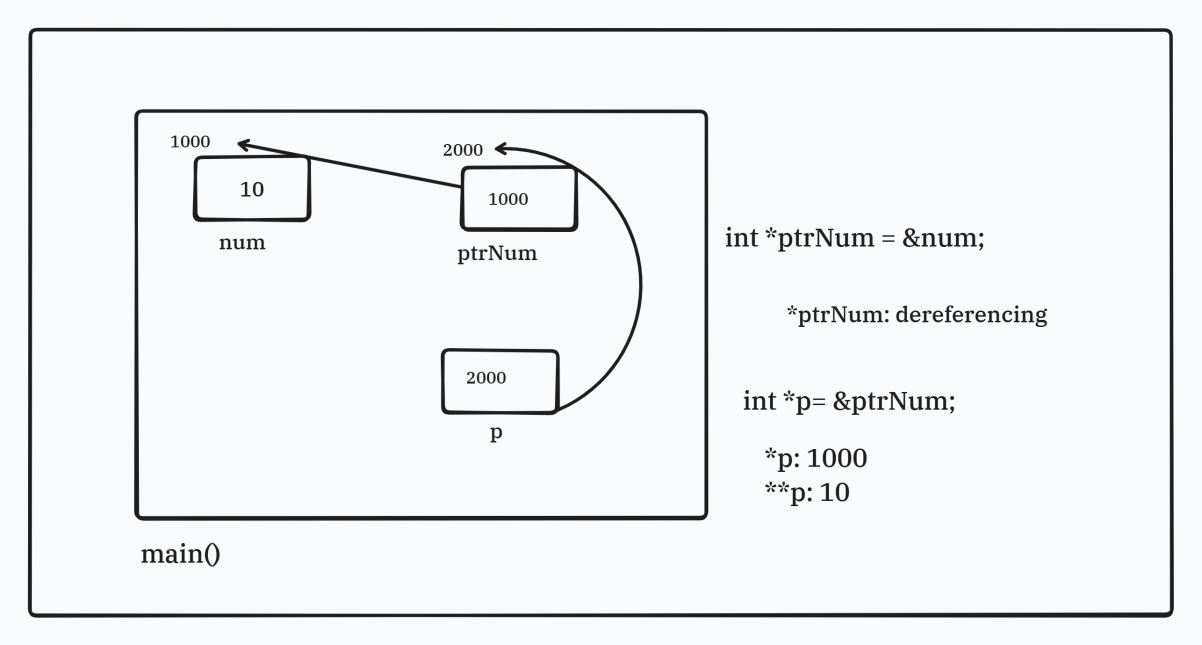
Pointers

Pointer is a variable which stores an address of another variable.

```
int val = 10; // val is a normal variable which stores an integer values
int *ptr = &val; // ptr is a pointer variable which stores the address of val variable
```

- Valid Syntax:

```
int* ptr;

int * ptr;

int *ptr;
```



NULL vs nullptr

- NULL is a Macro which stores 0 in it that is a int value.
- nullptr is pointer type which denoted null pointer.

> nullptr is a keyword introduced in C++11 to represent a null pointer. It is of type std::nullptr_t and can be implicitly converted to any pointer type, making it safer and more type-safe than NULL, which is typically defined as 0 or ((void*)0) in C. Using nullptr helps prevent ambiguity and potential errors when dealing with pointers in C++.

```c
#include <stdio.h>

void func(int val){
    printf("inside int func!");
}

void func(int* val){
    printf("inside int* func!");
}

int main() {

    func(nullptr);      // NULL: is a macro which stores int value 0 in it.

    int* pt = nullptr; // type: nullprt_t

    int* ptr = NULL;

    return 0;
}
```

const & pointer Combinations

const is a keyword that can be used with variable in order to make it read-only/constant.

```c
const int val = 10; // variable cannot be changed or reassigned

val = 20;   //error: assignment of read-only variable 'val'

printf("val: %d", val);
```

Comnbinations of const with pointer:

- const int* ptr;
- int* const ptr;
- const int* const ptr;

The above are important combinations of const with pointer.

These is important to understand as they are used in real world programming and also in OOP concepts.

const int* ptr

```c
#include <stdio.h>

int main(){

    int val = 10;
```

```
    //constant pointer
    const int *ptr = &val;

    int num = 20;

    ptr = &num; // valid

//  *ptr = 20;  // invalid- error: assignment of read-only location '*ptr'

    printf("ptr: %p", ptr);

    return 0;
}
```

int* const ptr

```
int main(){

    int val = 10;

    int* const ptr = &val;
//  int* ptr1 const;  invalid syntax

    *ptr = 20;  // valid

    int num = 20;

//  ptr = &num; // invalid- error: assignment of read-only variable 'ptr'

    printf("val: %d", val);

    return 0;

}
```

const int* const ptr

```
#include <stdio.h>

int main(){

    int val = 10;

    const int* const ptr = &val;

//  const * const int ptr;  //invalid syntax

//  *ptr = 20;  // invalid- error: assignment of read-only location '*(const
```

```
int*)ptr'
    int num = 20;

//  ptr = &num; // invalid- error: assignment of read-only variable 'ptr'

    printf("val: %d", val);

    return 0;
}
```

Limitations of Procedural Programming

- Everything is written in global context/area
- No data grouping concept
- No real world modeling concept
- No code reusability concept
- No data hiding concept
- No security concept

```
void displayStudent(int student_prn, int student_marks){
    printf("PRN: %d, Marks: %d", student_prn, student_marks);
}

int main(){
    int student_prn = 12345;
    int student_mark = 90;

    displayStudent(student_prn, student_mark);

    //if there are 100 students then we have to write 100 variables and also pass
100 variables to displayStudent function which is not a good approach.

    //or we can use arrays.
    int student_prns[100];
    int student_marks[100];

    displayStudent(student_prns[0], student_marks[0]);

    for(int i=0; i<100; i++){
        displayStudent(student_prns[i], student_marks[i]);
    }

    return 0;
}
```

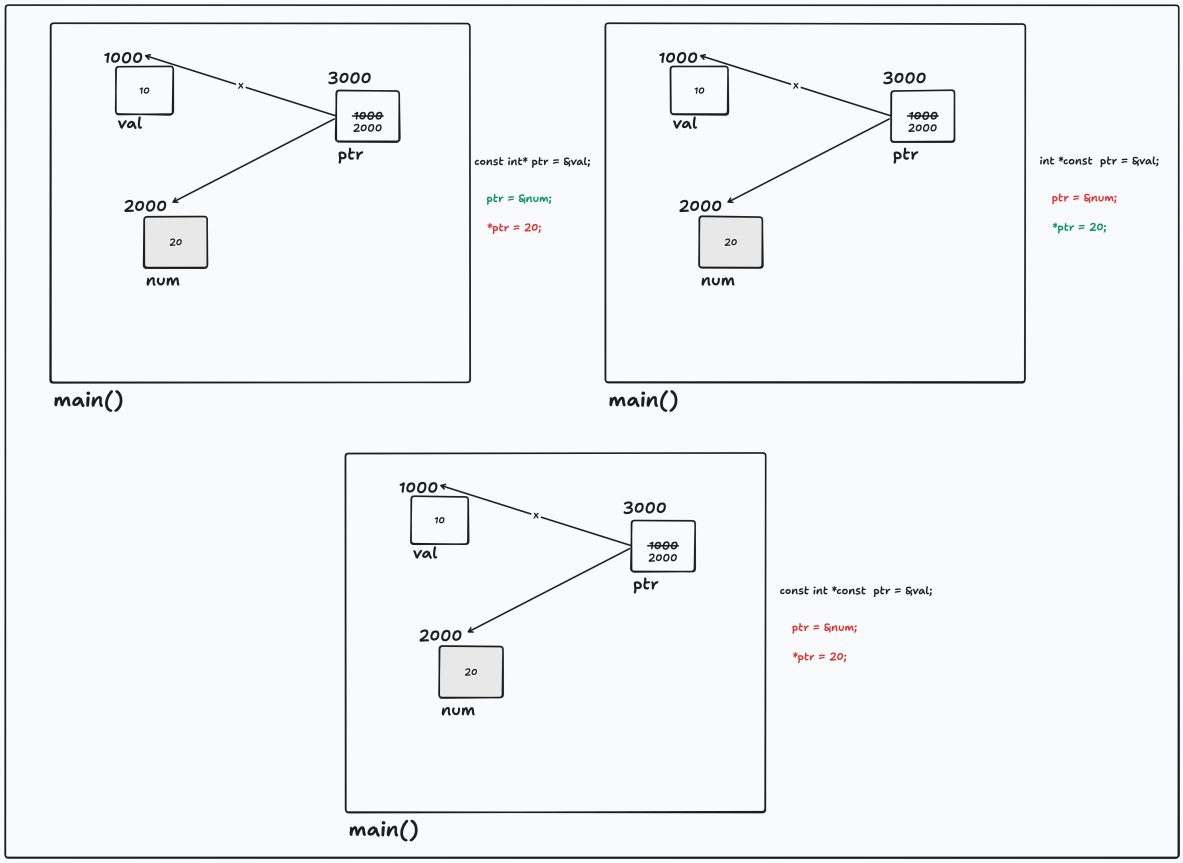
- The above code is not a good approach as it is not efficient and also it is not a good way to model real world entities like student, employee etc.
- Also, it is not a good way to group data together.
- Also, it is not a good way to reuse code.

Call by Value vs Call by Reference vs Call by Address

- Call by Value: In this method, a copy of the actual parameter is passed to the
- Call by Reference: In this method, the reference of the actual parameter is passed to the function.
- Call by Address: In this method, the address of the actual parameter is passed to the function.

```
#include <iostream>

struct Student{
    int rollNo;
    char name[50];
    float marks;
};  // student ends here

// call by value
void acceptData(struct Student s){
    printf("Enter Roll NO:");
    scanf("%d", &s.rollNo);
    printf("Enter Name:");
    scanf("%s", s.name);
    printf("Enter Marks:");
    scanf("%f", &s.marks);
}

void displayData(struct Student s){
    printf("\n Student: RollNO: %d, Name: %s, Marks: %f", s.rollNo, s.name,
s.marks);
}

// call by reference
void acceptData(struct Student &s){
    printf("Enter Roll NO:");
    scanf("%d", &s.rollNo);
    printf("Enter Name:");
    scanf("%s", s.name);
    printf("Enter Marks:");
    scanf("%f", &s.marks);
}

void displayData(struct Student &s){
    printf("\n Student: RollNO: %d, Name: %s, Marks: %f", s.rollNo, s.name,
s.marks);
}

// call by address
void acceptData(struct Student *s){
    printf("Enter Roll NO:");
    scanf("%d", &s->rollNo);
    printf("Enter Name:");
    scanf("%s", s->name);
    printf("Enter Marks:");
    scanf("%f", &s->marks);     //s->marks :(*s).marks
```

```
}

void displayData(struct Student *s){
    printf("\n Student: RollNO: %d, Name: %s, Marks: %f", s->rollNo, s->name, s-
>marks);
}

int main(){
    struct Student s1;

    //acceptData(rollno, name, marks);

    // call by value
    acceptData(s1);
    displayData(s1);

    // call by reference
    acceptData(s1);
    displayData(s1);

    // call by address
    acceptData(&s1);
    displayData(&s1);

    return 0;
}
```



Real-World Modeling Concept

- In real world, we have entities like student, employee, car, etc. which have certain attributes and behaviors.

> Model: It is a real world entity e.g. Student, Employee.

Data Grouping using struct in C

- struct is a user defined data type in C which is used to group different types of data together.
- It is used to model real world entities like student, employee, car, etc.
- It is also used to group data together.

```
struct Student{
    int rollNo;
    char name[50];
    float marks;
}

int main(){
    struct Student students[100];

    Student s1;

    s1.rollNo

    return 0;
}
```

struct in C vs struct in C++

- In C, struct cannot have member functions but in C++ struct can have member functions.
- In C, struct members are public by default
- In C, struct is used to group data together but in C++ struct is used to model real world entities and also to group data together.

```
//c style
struct Student{
    int rollNo;
    char name[50];
    float marks;
};

// c++ style
struct Student{
    int rollNo;
    char name[50];

    void displayData(){
    printf("\n Student: RollNO: %d, Name: %s, Marks: %f", s.rollNo, s.name,
s.marks);
```

```
}
};
```

Access Specifiers (public, private, protected)

- Access specifiers are used to achieve data hiding and encapsulation in OOP.
- Access specifiers are used to specify the access level of the members.
- One level of Abstraction can be achieved using access specifiers.
- **Abstraction** is the process of hiding the implementation details and showing only the functionality to the user. It is one of the fundamental concepts of OOP.

Access specifiers are mainly of three types:

- **public**: members are accessible from outside the class
- **private**: members are accessible only within the class
- **protected**: members are accessible within the class and also in derived class